

```

#include <iostream>
using namespace std;

// Base class for all logic gates
class Gate {
public:
    Gate *input[2] = {nullptr, nullptr}; // Array to store input gates (max 2 inputs)
    virtual bool output() = 0; // Pure virtual function to calculate the output
    virtual void setValue(Gate *gate, int pin); // Virtual function to set input gate
    virtual void setValue(bool, int); // Virtual function to set input value
};

// Derived class for the TRUE gate
class TRUE : public Gate {
public:
    bool output() override { return true; } // Returns true
    void setValue(Gate *, int) override {} // No operation for setting input gate
    void setValue(bool, int) override {} // No operation for setting input value
};

// Derived class for the FALSE gate
class FALSE : public Gate {
public:
    bool output() override { return false; } // Returns false
    void setValue(Gate *, int) override {} // No operation for setting input gate
    void setValue(bool, int) override {} // No operation for setting input value
};

TRUE t; // Instance of TRUE gate
FALSE f; // Instance of FALSE gate

// Set input value (true/false) for the Gate
void Gate::setValue(bool val, int pin) {
    if (val) this->input[pin] = &t; // Set input to TRUE gate if val is true
    else this->input[pin] = &f; // Set input to FALSE gate if val is false
}

// Set input gate for the Gate
void Gate::setValue(Gate *gate, int pin) { this->input[pin] = gate; }

// Derived class for the NOT gate
class NOT : public Gate {
public:
    bool output() override { return !(this->input[0]->output()); } // Returns the negation of the input
    void setValue(bool val, int pin = 0) override {
        if (val) this->input[0] = &t; // Set input to TRUE gate if val is true
        else this->input[0] = &f; // Set input to FALSE gate if val is false
    }
    void setValue(Gate *gate, int pin = 0) override { this->input[0] = gate; } // Set input gate
};

// Derived class for the NAND gate
class NAND : public Gate {
public:
    bool output() override { return !(this->input[0]->output() && this->input[1]->output()); } // Returns
the NAND of the inputs
    void setValue(Gate *gate, int pin) override { this->input[pin] = gate; } // Set input gate
    void setValue(bool val, int pin) override {
        if (val) this->input[pin] = &t; // Set input to TRUE gate if val is true
        else this->input[pin] = &f; // Set input to FALSE gate if val is false
    }
};

```

```

    }
};

// Derived class for the NOR gate
class NOR : public Gate {
public:
    bool output() override { return !(this->input[0]->output() || this->input[1]->output()); } // Returns
the NOR of the inputs
    void setValue(Gate *gate, int pin) override { this->input[pin] = gate; } // Set input gate
    void setValue(bool val, int pin) override {
        if(val) this->input[pin] = &t; // Set input to TRUE gate if val is true
        else this->input[pin] = &f; // Set input to FALSE gate if val is false
    }
};

// Derived class for the AND gate
class AND : public Gate {
public:
    AND() {
        component[0] = new NAND(); // Use a NAND gate
        component[1] = new NOT(); // Use a NOT gate
    }
    bool output() override {
        component[0]->input[0] = this->input[0]; // Set inputs of NAND gate
        component[0]->input[1] = this->input[1];
        component[1]->input[0] = component[0]; // Set input of NOT gate to the output of NAND gate
        return component[1]->output(); // Return the output of the NOT gate
    }
    void setValue(Gate *gate, int pin) override { this->input[pin] = gate; } // Set input gate
    void setValue(bool val, int pin) override {
        if (val) this->input[pin] = &t; // Set input to TRUE gate if val is true
        else this->input[pin] = &f; // Set input to FALSE gate if val is false
    }
private:
    Gate *component[2]; // Array to store component gates
};

// Derived class for the OR gate
class OR : public Gate {
public:
    OR() {
        component[0] = new NOR(); // Use a NOR gate
        component[1] = new NOT(); // Use a NOT gate
    }
    bool output() override {
        component[0]->input[0] = this->input[0]; // Set inputs of NOR gate
        component[0]->input[1] = this->input[1];
        component[1]->input[0] = component[0]; // Set input of NOT gate to the output of NOR gate
        return component[1]->output(); // Return the output of the NOT gate
    }
    void setValue(Gate *gate, int pin) override { this->input[pin] = gate; } // Set input gate
    void setValue(bool val, int pin) override {
        if (val) this->input[pin] = &t; // Set input to TRUE gate if val is true
        else this->input[pin] = &f; // Set input to FALSE gate if val is false
    }
private:
    Gate *component[2]; // Array to store component gates
};

// Abstract base class for Mux (Multiplexer)

```

```

class Mux {
public:
    virtual void setData(bool, int) = 0; // Pure virtual function to set data input
    virtual void setData(Gate *, int) = 0; // Pure virtual function to set data input gate
    virtual void setEnable(bool) = 0; // Pure virtual function to set enable signal
    virtual void setEnable(Gate *) = 0; // Pure virtual function to set enable signal gate
    virtual void setSelect(bool, int) = 0; // Pure virtual function to set select signal
    virtual void setSelect(Gate *, int) = 0; // Pure virtual function to set select signal gate
    virtual Gate *output() = 0; // Pure virtual function to get output gate
protected:
    Gate *enable; // Pointer to enable gate
};

// Class for 2-to-1 Multiplexer
class Mux2_1 : public Mux {
public:
    Mux2_1() : Mux2_1(0) {} // Constructor with default value
    Mux2_1(bool val) { // Constructor with parameter
        if(val) this->enable = &t; // Set enable to TRUE gate if val is true
        else this->enable = &f; // Set enable to FALSE gate if val is false
        component[0] = new AND; // Create an AND gate
        component[1] = new AND; // Create another AND gate
        component[2] = new NOT; // Create a NOT gate
        component[3] = new OR; // Create an OR gate
        component[4] = new AND; // Create another AND gate
    }
    void setData(bool val, int pin) override {
        switch(pin) {
            case 0 : component[0]->setValue(val, 0); break; // Set data input for first AND gate
            case 1 : component[1]->setValue(val, 0); break; // Set data input for second AND gate
            default : ;
        }
    }
    void setData(Gate *gate, int pin) override {
        switch(pin) {
            case 0 : component[0]->setValue(gate, 0); break; // Set data input gate for first AND gate
            case 1 : component[1]->setValue(gate, 0); break; // Set data input gate for second AND
gate
            default : ;
        }
    }
    void setEnable(bool val) override {
        if(val) this->enable = &t; // Set enable to TRUE gate if val is true
        else this->enable = &f; // Set enable to FALSE gate if val is false
    }
    void setEnable(Gate *gate) override {
        this->enable = gate; // Set enable gate
    }
    void setSelect(bool val, int pin = 0) override {
        component[1]->setValue(val, 1); // Set select input for second AND gate
        component[2]->setValue(val, 0); // Set select input for NOT gate
        component[0]->input[1] = component[2]; // Set second input of first AND gate to NOT gate
output
    }
    void setSelect(Gate *gate, int pin = 0) override {
        component[1]->setValue(gate, 1); // Set select input gate for second AND gate
        component[2]->setValue(gate, 0); // Set select input gate for NOT gate
        component[0]->input[1] = component[2]; // Set second input of first AND gate to NOT gate
output
    }
}

```

```

    Gate *output() override {
        component[3]->setValue(component[0], 0); // Set first input of OR gate to first AND gate
    output
        component[3]->setValue(component[1], 1); // Set second input of OR gate to second AND
    gate output
        component[4]->setValue(this->enable, 0); // Set first input of final AND gate to enable signal
        component[4]->setValue(component[3], 1); // Set second input of final AND gate to OR gate
    output
        return component[4]; // Return final AND gate
    }
private:
    Gate *component[5]; // Array to store component gates
};

```

// Class for 4-to-1 Multiplexer

```

class Mux4_1 : public Mux {
public:
    Mux4_1() {} // Default constructor
    Mux4_1(bool val) {
        mux2_1[0] = new Mux2_1(val); // Create first 2-to-1 Mux
        mux2_1[1] = new Mux2_1(val); // Create second 2-to-1 Mux
        mux2_1[2] = new Mux2_1(val); // Create third 2-to-1 Mux
    }
    void setData(bool val, int pin) override {
        if (pin < 2)
            mux2_1[0]->setData(val, pin); // Set data for first 2-to-1 Mux
        else
            mux2_1[1]->setData(val, pin - 2); // Set data for second 2-to-1 Mux
    }
    void setData(Gate *gate, int pin) override {
        if (pin < 2)
            mux2_1[0]->setData(gate, pin); // Set data gate for first 2-to-1 Mux
        else
            mux2_1[1]->setData(gate, pin - 2); // Set data gate for second 2-to-1 Mux
    }
    void setEnable(bool val) override {
        for (int i = 0; i < 3; ++i)
            mux2_1[i]->setEnable(val); // Set enable signal for all 2-to-1 Muxes
    }
    void setEnable(Gate *gate) override {
        for (int i = 0; i < 3; ++i)
            mux2_1[i]->setEnable(gate); // Set enable gate for all 2-to-1 Muxes
    }
    void setSelect(bool val, int pin) override {
        if (pin == 0) {
            mux2_1[0]->setSelect(val, 0); // Set select signal for first 2-to-1 Mux
            mux2_1[1]->setSelect(val, 0); // Set select signal for second 2-to-1 Mux
        } else {
            mux2_1[2]->setSelect(val, 0); // Set select signal for third 2-to-1 Mux
        }
    }
    void setSelect(Gate *gate, int pin) override {
        if (pin == 0) {
            mux2_1[0]->setSelect(gate, 0); // Set select gate for first 2-to-1 Mux
            mux2_1[1]->setSelect(gate, 0); // Set select gate for second 2-to-1 Mux
        } else {
            mux2_1[2]->setSelect(gate, 0); // Set select gate for third 2-to-1 Mux
        }
    }
    Gate *output() override {

```

```

        mux2_1[2]->setData(mux2_1[0]->output(), 0); // Set first input of third 2-to-1 Mux to output of
first 2-to-1 Mux
        mux2_1[2]->setData(mux2_1[1]->output(), 1); // Set second input of third 2-to-1 Mux to
output of second 2-to-1 Mux
        return mux2_1[2]->output(); // Return output of third 2-to-1 Mux
    }
private:
    Mux *mux2_1[3]; // Array to store 2-to-1 Muxes
};

// Class for XOR gate implemented using Mux
class XOR_Mux : public Gate {
public:
    XOR_Mux() : mux(true) { // Constructor initializing Mux with true
        mux.setEnable(true); // Set Mux enable signal to true
    }
    void setValue(bool val, int pin) override {
        mux.setSelect(val, pin); // Set select signal for Mux
    }
    void setValue(Gate *gate, int pin) override {
        mux.setSelect(gate, pin); // Set select gate for Mux
    }
    bool output() override {
        mux.setData(false, 0); // Set data inputs for Mux to implement XOR logic
        mux.setData(true, 1);
        mux.setData(true, 2);
        mux.setData(false, 3);
        return mux.output()->output(); // Return the output of the Mux
    }
private:
    Mux4_1 mux; // Instance of 4-to-1 Mux
};

int main()
{
    XOR_Mux xorGate; // Create an XOR gate using Mux

    int a, b;
    cin >> a >> b; // Read two input values

    xorGate.setValue(a, 0); // Set first input of XOR gate
    xorGate.setValue(b, 1); // Set second input of XOR gate

    cout << xorGate.output() << endl; // Output the result of XOR gate

    return 0; // Return success
}

```